



PARALLEL AND DISTRIBUTED COMPUTING

8 – esercitazione MPI

Marco Lapegna
Dipartimento di Matematica e Applicazioni
Università degli Studi di Napoli Federico II

www.docenti.unina.it/marco.lapeгна

1

Marco Lapegna
Parallel and Distributed Computing
8 – esercitazione MPI

prima cosa: l'educazione

scrivere ed eseguire sul cluster didattico il programma che produce il seguente output

```
%  
hello from 1 di 4 processi  
hello from 2 di 4 processi  
hello from 0 di 4 processi  
hello from 3 di 4 processi  
%
```

2

compilare ed eseguire programmi MPI

compilazione

```
mpicc sorgente1.c sorgente2.c sorgente3.c -o executable
```

nomi file sorgente nome file eseguibile

il comando **mpicc** compila i file sorgente e li collega alla libreria **MPI**

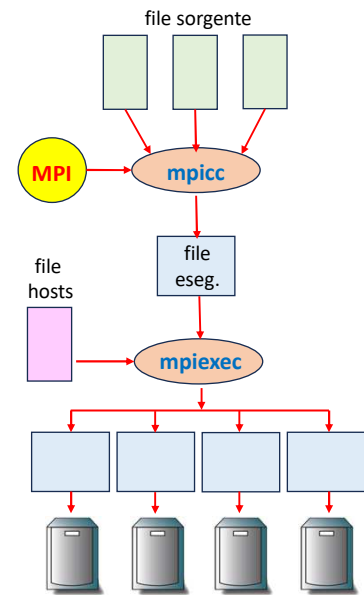
esecuzione

```
mpiexec -np numproc -machinefile hosts executable
```

numero di processi MPI da attivare file con i nomi dei nodi del cluster da usare nome file eseguibile

Il comando **mpiexec** manda in esecuzione **numproc** volte l'eseguibile **executable** sui nodi del cluster elencati in **host**

la directory del cluster didattico dove risiedono i comandi
/usr/lib64/openmpi/1.4-gcc/bin



3

integrazioni al file pbs

```
#!/bin/bash

#PBS -q studenti
#PBS -l nodes=8:ppn=8
#PBS -l mem=1GB
#PBS -N hello
#PBS -o hello.out
#PBS -e hello.err

cat $PBS_NODEFILE
cd $PBS_O_WORKDIR

sort -u $PBS_NODEFILE > machinefile
NCPU=`wc -l < machinefile`

echo "Il job e' in esecuzione sugli" $NCPU " nodi:"
cat machinefile

MPI_DIR=/usr/lib64/openmpi/1.4-gcc/bin
$MPI_DIR/mpicc -o hello hello.c
$MPI_DIR/mpiexec -np $NCPU -machinefile machinefile hello
```

questa opzione richiede al pbs l'utilizzo di 8 nodi, ciascuno con 8 unità di calcolo. Produce un file (**\$PBS_NODEFILE**) con i nomi dei nodi da utilizzare

facoltativo: per vedere cosa c'è in **\$PBS_NODEFILE**

ordina ed elimina ripetizioni in **\$PBS_NODEFILE**. produce il nuovo file **machinefile** e determina il numero di nodi contandone le righe

facoltativo: per vedere numero e nomi dei nodi utilizzati

compilazione ed esecuzione

4

scrivere una function MPI

```
int somma(int *A , int N)
```

che calcola la somma totale degli elementi degli array **A** di lunghezza **N** in ogni processo (strategia: ring sum)

In ogni processo, il programma chiamante definisce gli array e stampa il risultato totale

Ad es. con **NP=4** processi e **N=10**, inizializzare gli array

```
A[i] = myid*10 + i
```

myid=0
A=0, ..., 9

myid=1
A=10, ..., 19

myid=2
A=20, ..., 29

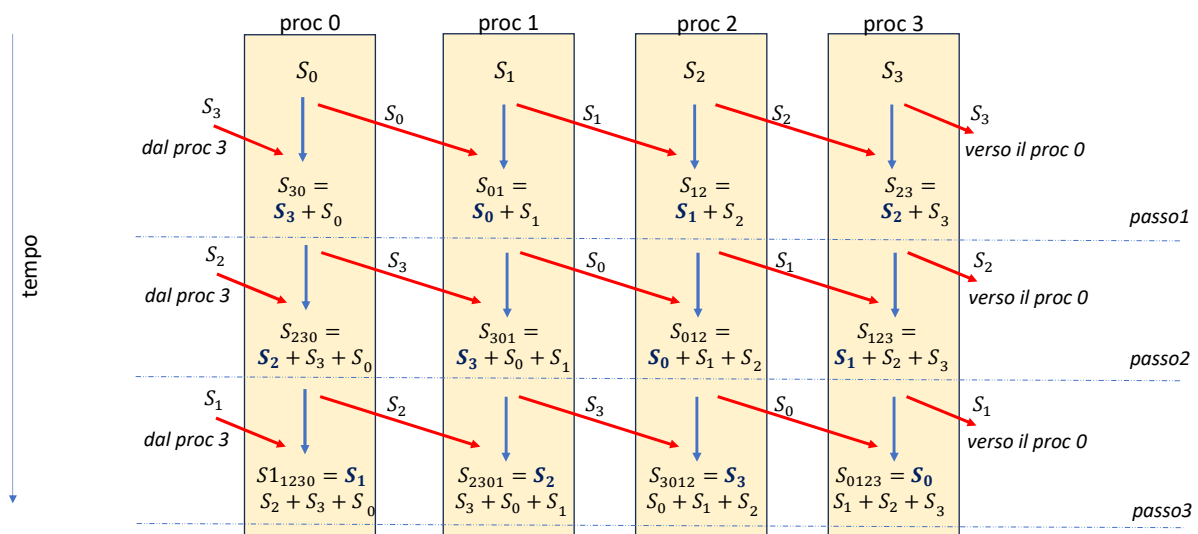
myid=3
A=30, ..., 39

La somma totale è **780**

sommampi.c

5

ogni processo spedisce al processo successivo un dato e ne riceve un altro dal processo precedente



necessari 3 passi (in generale **NP** passi, con **NP**=numero dei processi)

6

Algoritmo SPMD
stesso algoritmo in esecuzione
concorrentemente in tutti i processi

```

. . .
Stot = S
token = S
for i = 1 to NP-1
    send(token, (myid + NP + 1)%NP )
    recv(token, (myid + NP - 1)%NP )
    Stot = Stot + token
endfor
. . .

```

- **myid** identificativo del processo
- **NP** numero di processi
- **S** risultato parziale del processo
- **Stot** risultato totale
- **token** valore che viene ricevuto e passato da ogni processo

Osservazione:

tutti i processi possiedono il risultato totale

7

in ogni processo, inizializzare l'array come **$A[i] = \text{myid} \cdot 10 + i$** ($i=0, \dots, 10$)

ad esempio, con **NP = 8** processi

P_0	0	1	2	3	4	5	6	7	8	9
P_1	10	11	12	13	14	15	16	17	18	19
P_2	20	21	22	23	24	25	26	27	28	29
P_3	30	31	32	33	34	35	36	37	38	39
P_4	40	41	42	43	44	45	46	47	48	49
P_5	50	51	52	53	54	55	56	57	58	59
P_6	60	61	62	63	63	65	66	67	68	69
P_7	70	71	72	74	74	75	76	77	78	79

Stot = 3160

8